UNIVERSITY EXAM RESULT PORTAL USING READER–WRITER SYNCHRONIZATION WITH GUI AND SYSV IPC

Student Information

Name: Muhammad Ali
Roll Number: 23K-0015
Course: Operating Systems
Semester: Spring 2026

1. Objective

The objective of this project is to design and implement a university exam result portal that demonstrates core Operating System concepts, especially synchronization and inter-process communication. The system provides separate interfaces for students and administrators and ensures correct concurrent access to shared data using Reader–Writer logic.

Specifically, the project aims to:

- Allow multiple student read operations safely.
- Enforce exclusive access for admin write operations.
- Use low-level system calls:
  - `fork()`
  - `pipe()`
  - `read()`
  - `write()`
  - `open()`
  - `close()`
- Use System V IPC mechanisms:
  - `shmget()`

    - `shmat()`
    - `shmdt()`
    - `shmctl()`
    - `semget()`
    - `semop()`
- Maintain consistent file-based storage for results and credentials.

2. Introduction

During result announcements, a university portal faces heavy concurrent traffic. Many students want to view results at the same time, while administrators may still need to update records. If such access is not synchronized, race conditions and inconsistent data can occur.

To model this real-world scenario, this project implements a **Reader–Writer based result management system** where:

- Students act as **readers**.
- Administrators act as **writers**.
- Readers may access data concurrently when safe.
- Writers get exclusive access for updates.

The project combines a GUI front-end with multi-process backend processing to reflect practical systems programming rather than only a simple command-line simulation.

3. Background

The **Reader–Writer Problem** is a classical synchronization problem in Operating Systems. It focuses on balancing high read concurrency with safe write exclusivity. Different scheduling policies are possible, including:

- **Reader Preference**
- **Writer Preference**
- **Fair Scheduling**

This implementation uses semaphore-based coordination with shared state for correct behavior across processes.

The project architecture uses:

- Worker process creation with `fork()`
- Request/response communication through `pipe()` and `read()/write()`
- Shared metadata in **System V shared memory**
- Semaphore operations for synchronization
- Raw file I/O for persistent records

A **GTK-based minimal GUI** is implemented with separate **Student View** and **Admin View**, matching the required role-based interface design.

4. Platform and Languages

Operating System

Ubuntu Linux (VirtualBox environment)

Programming Language

C (C11)

GUI Toolkit

GTK+ 3

Build System

Makefile

Compiler

GCC

Key APIs and System Calls

Process Management

- `fork()`
- `waitpid()`

Inter-Process Communication (IPC)

- `pipe()`
- `read()`
- `write()`

Shared Memory (System V IPC)

- `shmget()`
- `shmat()`
- `shmdt()`
- `shmctl()`

Semaphores (System V IPC)

- `semget()`
- `semop()`
- `semctl()`

File I/O

- `open()`
- `read()`
- `write()`
- `close()`
- `rename()`

5. Methodology

5.1 System Design

The project is divided into the following modules:

GUI Module (`gtk_gui.c`)

Provides tab-based Student and Admin interfaces.

Core Backend Module (`reader_writer.c`)

Implements synchronization, authentication, result retrieval, and record updates.

Request Coordinator (`main.c`)

Demonstrates process-based execution and worker handling for requests.

Shared Definitions (`reader_writer.h`)

Contains request/response structures and shared-state definitions.

5.2 Workflow

1. User selects role (**Student** or **Admin**) in the GUI.
2. GUI collects input fields.
3. GUI creates a request structure and sends it to a forked worker process via pipes.
4. Worker processes the request using synchronized access rules:
    a. Readers enter the reader section.
    b. Writers acquire exclusive resource lock.
5. Worker returns response text to GUI.
6. GUI displays the output to the user.

5.3 Data Handling

The system uses the following files:

- **admins.txt** → Stores administrator credentials
- **passwords.txt** → Stores student credentials
- **results.txt** → Stores student marks records
- **results.tmp** → Temporary file for safe result updates
- **passwords.tmp** → Temporary file for safe credential updates
- **operations.log** → Stores operation logs with timestamps

Temporary files are replaced using `rename()` to ensure safer file consistency.

5.4 Synchronization Strategy

Shared memory stores counters such as:

- Active reader count

- Total read operations
- Total write operations

System V semaphores control:

- Queue order
- Reader-count update region
- Exclusive resource lock for writers

This prevents:

- Write-write conflicts
- Read-write conflicts
- Race conditions

6. Results (Graphs if there is comparison)

6.1 Functional Results

The implemented system successfully provides:

- Separate Student/Admin GUI views
- Student authentication and result retrieval
- Admin authentication and add/update operations
- Synchronized concurrent request handling
- Low-level IPC and shared-memory based processing

6.2 Suggested Performance Comparison

For report evaluation, include a small comparison between **read-heavy** and **write-heavy** workloads.

Use the following table format in Word:

Test Case	Student Requests	Admin Requests	Notes
Read-heavy	20	2	High concurrent reads

Balanced	10	10	Mixed load
Write-heavy	5	15	More exclusive writes

6.3 Graphs to Insert

Create and insert the following graphs in Word:

Bar Graph

- **X-axis:** Read-heavy, Balanced, Write-heavy
- **Y-axis:** Average Response Time (ms)

Column Graph

- **X-axis:** Same test cases
- **Y-axis:** Total Reads and Total Writes (from shared counters/log runs)

Graph Caption Example

Figure X: Performance comparison under different workloads.

7. Conclusion

This project successfully demonstrates practical Operating System concepts by integrating synchronization theory with process-level implementation. The system provides a functional role-based GUI and enforces safe concurrent access using Reader–Writer synchronization in a multi-process environment.

Major Achievements

- Use of explicit low-level system calls
- Use of System V shared memory and semaphores
- Reliable role separation and request handling
- Consistent result storage with safe update flow

Overall, the project bridges academic Operating System concepts and real systems programming practice. The system can be extended further with:

- Richer analytics
- Stronger security mechanisms
- Advanced GUI improvements